

## Lecture 18: Unsupervised Learning

*Instructor: Jonathan Pipping-Gamón**Authors: RB, JP*

## 18.1 Replacing Players in the NBA

Suppose an NBA team loses a player with a specific offensive role in free agency. As the team searches for a replacement, the useful first question is often not, “Who is the best available player?” but “Who has played a similar role?”

A second-unit pick-and-roll creator should be compared with guards who have handled similar on-ball usage. A low-usage center who screens, rolls, cuts, and finishes belongs in a different search pool. Our task is player similarity: find historical player-seasons with play-type mixes close to a target player-season.

This is not yet a prediction problem. There is no response variable, no known “correct” replacement, and no outcome  $y$  to predict. We only have features describing how players play. That is the setting for unsupervised learning.

The data are NBA Synergy-style play-type proportions from 2012–2022. Each row is a player-season, and each feature is the share of that player’s attempts from a given play type:

- spot-up
- pick-and-roll ball handler
- pick-and-roll roller
- post-up
- cut
- offensive rebound
- isolation
- off-screen or handoff

Let  $\mathbf{x}_i \in \mathbb{R}^8$  denote the play-type vector for player-season  $i$ . Two player-seasons are similar when their vectors describe similar offensive usage patterns.

## 18.2 Unsupervised Learning Starts Without Labels

In supervised learning, we observe a response  $y$  and learn a function from features to outcomes:

$$y = f(\mathbf{X}).$$

Regression, classification, and forecasting usually have this structure.

In unsupervised learning, there is no observed response. We only have the feature matrix  $\mathbf{X}$ . The goal is not prediction; it is structure discovery.

Today we focus on two tasks:

- **Dimensionality reduction:** construct a lower-dimensional representation that preserves important variation.
- **Clustering:** group observations with similar feature profiles.

In the NBA example, both tasks use the same feature matrix. PCA gives a two-dimensional map of player-seasons. Clustering groups player-seasons with similar play-type profiles.

### 18.3 Player Vectors and Distance

Comparing players' play-type vectors becomes meaningful only after we define similarity. A common choice is Euclidean distance:

$$d(\mathbf{x}_i, \mathbf{x}_{i'}) = \|\mathbf{x}_i - \mathbf{x}_{i'}\|_2 = \sqrt{\sum_{j=1}^p (x_{ij} - x_{i'j})^2}.$$

Small distance means the two player-seasons are close on the selected features.

Since most metrics are sensitive to the scale of the data, we often standardize features before computing distances:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j},$$

where  $\bar{x}_j$  and  $s_j$  are the mean and standard deviation of feature  $j$ . Without standardization, a high-variance feature can dominate the distance calculation even when it is not more important for our question.

This step is a modeling choice. In unsupervised learning, **distance is the model**: the features, scaling, and metric define what counts as similar.

### 18.4 PCA Builds a Similarity Map

The NBA data set has eight features. That is small, but still hard to inspect directly. Principal component analysis (PCA) gives a lower-dimensional summary that preserves as much variation as possible.

Geometrically, PCA finds the directions where the point cloud varies most. As demonstrated in Figure 18.1, if the data form an oval, the first principal component follows the long axis. The second principal component is orthogonal to the first and captures the largest remaining direction of variation.

Let  $\mathbf{Z}$  be the standardized data matrix. PCA finds directions in feature space that maximize variance. The first principal component direction is

$$\mathbf{v}_1 = \arg \max_{\|\mathbf{v}\|_2=1} \text{Var}(\mathbf{Z}\mathbf{v}).$$

The second principal component solves the same problem subject to orthogonality:

$$\mathbf{v}_2 = \arg \max_{\|\mathbf{v}\|_2=1, \mathbf{v}^T \mathbf{v}_1=0} \text{Var}(\mathbf{Z}\mathbf{v}).$$

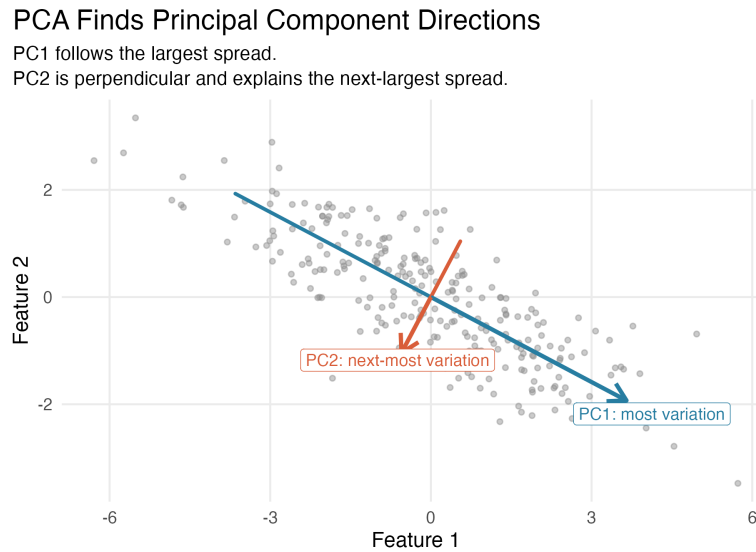


Figure 18.1: PCA finds directions of variation in the feature space. The first principal component follows the longest direction of the point cloud, and the second principal component is the next orthogonal direction.

The score for observation  $i$  on component  $\ell$  is

$$u_{i\ell} = \mathbf{z}_i^T \mathbf{v}_\ell.$$

The variance explained by component  $\ell$  is the variance of its scores:

$$\lambda_\ell = \text{Var}(\mathbf{Z}\mathbf{v}_\ell).$$

The proportion of total variance explained by that component is

$$\frac{\lambda_\ell}{\lambda_1 + \dots + \lambda_p}.$$

These proportions are what appear in PCA plots and scree plots: if PC1 explains 40%, then 40% of the standardized feature variation lies along the first principal component direction.

The vector  $\mathbf{v}_\ell$  is a loading vector. Large positive or negative loadings show which original features define a component.

In these data, PC1 separates paint-finishing bigs from perimeter creators. Roll-man, cut, and offensive-rebound usage load positively; pick-and-roll ball-handler, isolation, off-screen, and spot-up usage load negatively. PC2 separates spot-up shooting from isolation and pick-and-roll creation.

After computing principal component scores, we can plot every player-season in two dimensions. Nearby points have similar play-type profiles in this reduced representation. For a replacement search, nearby points are natural role comparisons. The map also shows whether the target player sits in a dense region of common roles or a sparse region with few close analogues.

### What Do the First Two Principal Components Measure?

Loadings from standardized NBA play-type proportions

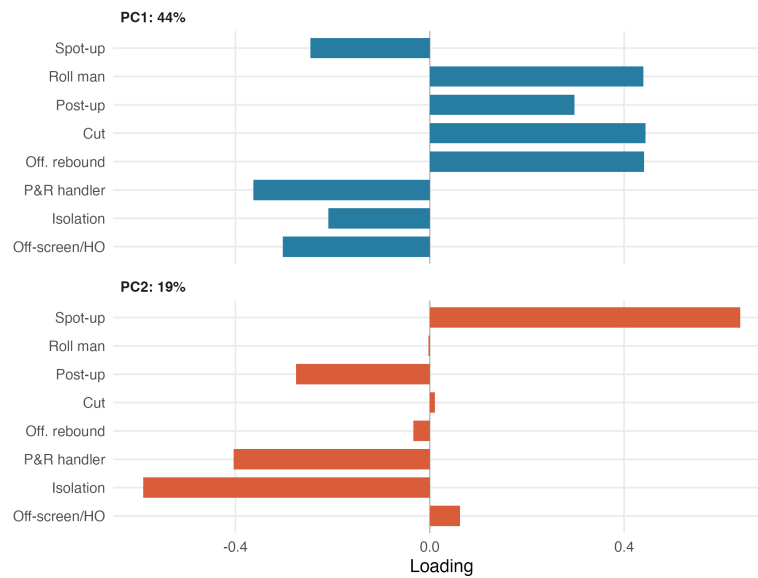


Figure 18.2: Loadings for the first two principal components of the NBA play-type data.

### A Two-Dimensional Map of NBA Shot Diets

PC1 explains 44% and PC2 explains 19% of standardized feature variation

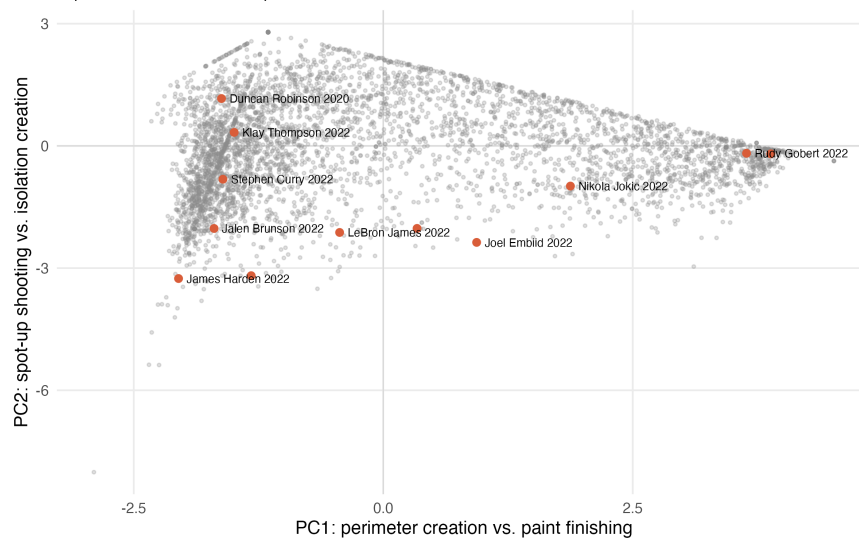


Figure 18.3: PCA map of NBA player-seasons using standardized play-type proportions.

## 18.5 Clustering Groups Similar Player-Seasons

Clustering partitions observations using their features. Given observations  $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ , a clustering algorithm assigns them to sets  $C_1, \dots, C_k$  so that observations within a cluster are similar and observations in different clusters are less similar.

A cluster is a similarity group. It is not automatically a basketball role, archetype, or recommendation. The algorithm returns assignments; we add meaning by inspecting the players and feature profiles in each group.

## 18.6 K-means Clustering

There are many clustering algorithms, but k-means is one of the most widely used. Choose a number of clusters  $k$ . The algorithm finds  $k$  centroids and assigns each observation to the nearest centroid.

The objective is to minimize within-cluster squared distance:

$$\min_{C_1, \dots, C_k} \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2,$$

where

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i$$

is the centroid of cluster  $C_j$ .

The standard algorithm alternates between two steps:

1. Assign each observation to its nearest current centroid.
2. Recompute each centroid as the mean of its assigned observations.

These steps decrease the objective until the assignments stop changing. Because the final solution can depend on centroid initialization, we usually run k-means from several random starts and keep the best solution.

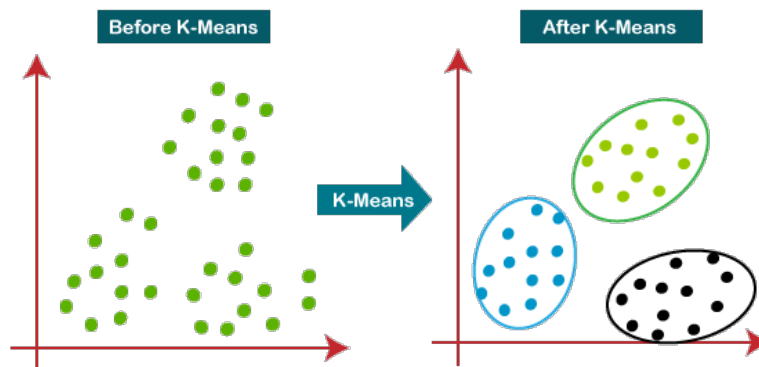


Figure 18.4: K-means alternates between assigning observations to centroids and moving centroids to cluster means.

K-means gives hard assignments: each player-season belongs to exactly one cluster. It also favors compact, roughly spherical clusters because it uses squared Euclidean distance to centroids.

## 18.7 Choosing the Number of Clusters

Ordinary k-means does not learn the number of clusters. We choose  $k$ . One common diagnostic is the elbow plot, which compares within-cluster sum of squares across candidate values of  $k$ .

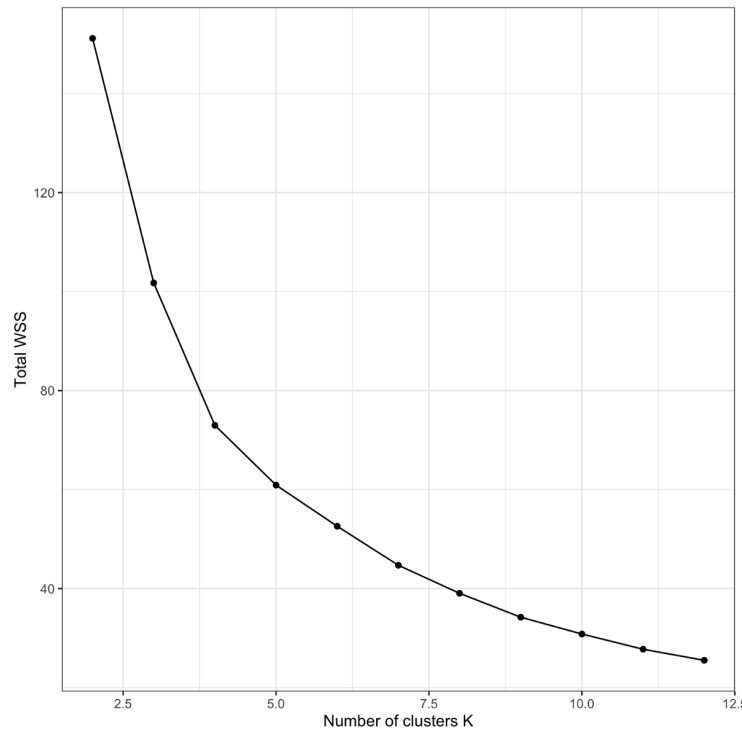


Figure 18.5: An elbow plot compares within-cluster variation for different values of  $k$ .

As  $k$  increases, within-cluster variation always decreases. The goal is a useful tradeoff: enough clusters to capture real structure, but not so many that small variations become separate groups. In practice, the elbow is often not decisive. It narrows the range; interpretation helps choose the final value. In the NBA player-seasons example, we fit a candidate  $k = 5$  and inspect each cluster's average play-type profile. Those five profiles split cleanly into recognizable role groups, so  $k = 5$  is useful for the roster question. The labels in Figure 18.6 are basketball interpretations we add after seeing the similarity groups.

### Five K-Means Clusters Split into NBA Roles

Average share of shot attempts by play type within each cluster ( $k = 5$ )

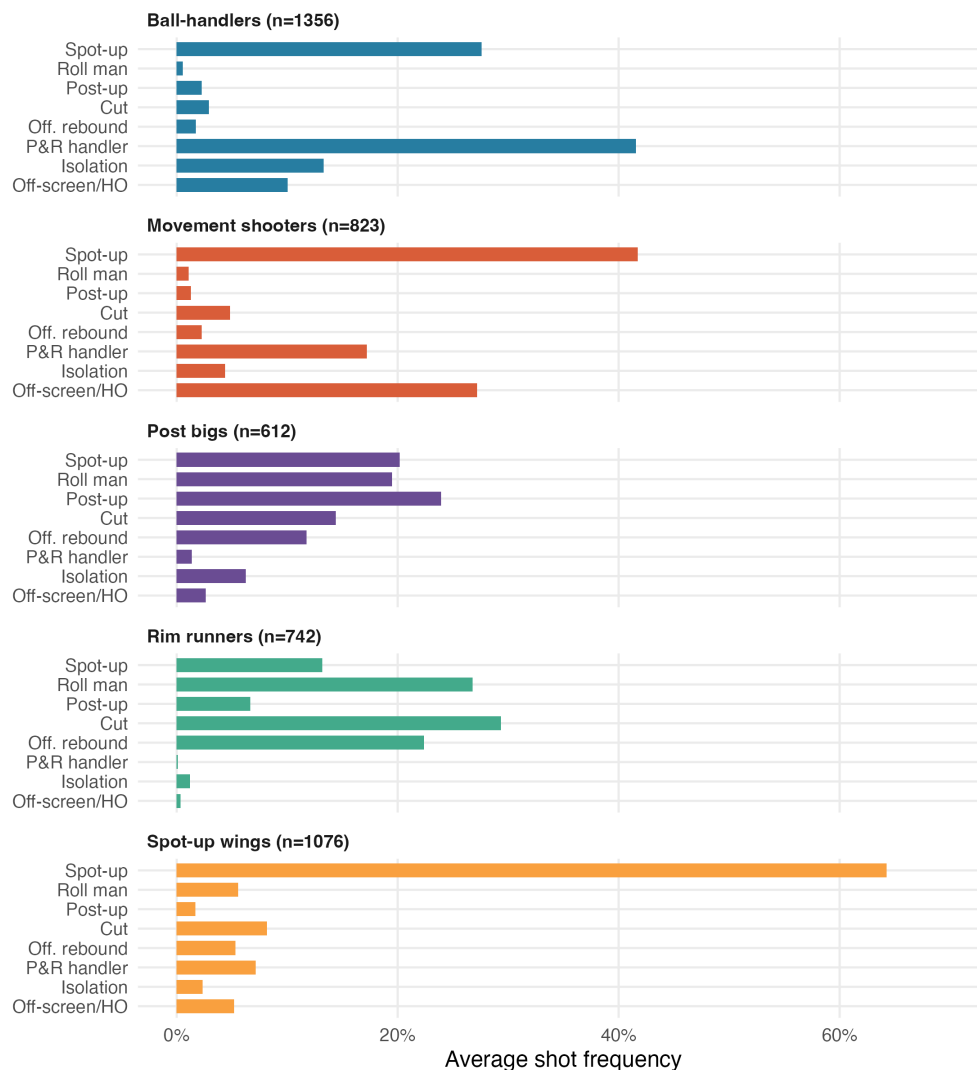


Figure 18.6: Average play-type mix within each of the five NBA player-season clusters. Each panel is one cluster's mean profile; the role name is an interpretation of that profile.

## 18.8 Replacing Jalen Brunson

We can now return to the roster question and turn the clusters into a candidate list. Suppose Dallas begins a replacement search after Jalen Brunson left following the 2021–22 season. Brunson's 2022 play-type profile places him in the **Ball-handlers** archetype: about 52% of his attempts came as a pick-and-roll ball handler, 17% from spot-ups, 17% from isolation, and 8% from off-screen or handoff actions.

To build an initial comparison list, compute the standardized Euclidean distance from Brunson's player-season vector to recent candidate player-seasons. Table 18.1 shows the nearest neighbors after restricting

the pool to 2021 and 2022 seasons and removing Brunson himself.

| Player-season         | Archetype     | Dist. | P&R | Iso | Spot-up |
|-----------------------|---------------|-------|-----|-----|---------|
| DeMar DeRozan 2021    | Ball-handlers | 0.49  | 46% | 18% | 16%     |
| DeMar DeRozan 2022    | Ball-handlers | 0.53  | 47% | 16% | 18%     |
| De'Aaron Fox 2021     | Ball-handlers | 0.56  | 53% | 14% | 20%     |
| De'Aaron Fox 2022     | Ball-handlers | 0.64  | 48% | 15% | 21%     |
| Zach LaVine 2022      | Ball-handlers | 0.65  | 45% | 16% | 22%     |
| Kevin Porter Jr. 2021 | Ball-handlers | 0.66  | 50% | 18% | 19%     |
| Kevin Porter Jr. 2022 | Ball-handlers | 0.71  | 49% | 20% | 19%     |
| Reggie Jackson 2021   | Ball-handlers | 0.72  | 48% | 18% | 24%     |

Table 18.1: Nearest recent player-seasons to Jalen Brunson 2022 by Euclidean distance in the standardized eight-feature play-type space. Candidates are restricted to the 2021 and 2022 seasons.

The table does not say these players match Brunson in quality, salary, age, health, or availability. It says only that their offensive usage vectors are close under the model we chose. The nearest rows are all ball-handler seasons, which is a useful sanity check that our algorithm is working as expected.

## 18.9 Checking an Unsupervised Result

Without labels, there is no single accuracy score. The question is whether the structure is coherent and useful for the decision. For clustering, check:

- **Interpretability:** do clusters have distinct feature profiles that we can describe in basketball terms?
- **Geometry:** are points closer to their own cluster than to other clusters? Which players sit near boundaries?
- **Stability:** do the same broad groups appear after changing seeds, perturbing the data, or trying nearby values of  $k$ ?
- **Usefulness:** does the result produce a candidate list worth investigating?

For player replacement, the final output should not be just a cluster label. Use the cluster as a broad role family, inspect the nearest player-seasons inside or near that family, then bring back basketball context: quality, salary, age, health, roster fit, and availability. Unsupervised learning narrows the search; it does not make the final decision.

## 18.10 Takeaways

- Unsupervised learning looks for structure in features without using a response variable.
- PCA turns high-dimensional player profiles into a lower-dimensional similarity map.
- Clustering groups similar observations, but the labels are human interpretations.
- K-means is simple and useful, but it requires choosing  $k$  and favors compact clusters.
- Similarity is a modeling choice: it depends on the features, the distance metric, and the interpretation after inspection.



## Appendix

### A Hierarchical Clustering

K-means starts by choosing  $k$ . Hierarchical clustering instead builds a tree of similarities, called a dendrogram. Agglomerative hierarchical clustering starts with each observation in its own cluster, then repeatedly merges the two closest clusters.

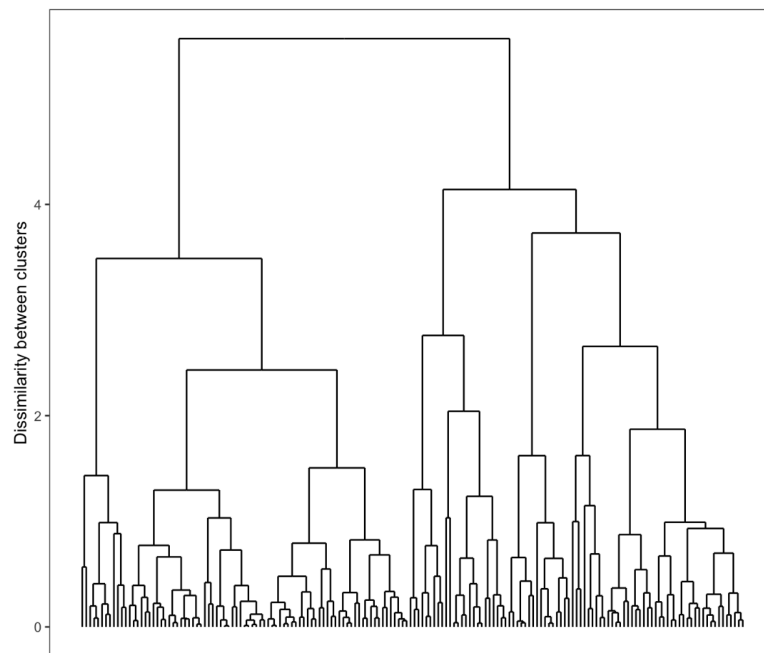


Figure 18.7: A dendrogram records which clusters merge and the dissimilarity level at which they merge.

The key modeling choice is linkage: how to define distance between clusters rather than between individual observations.

- **Single linkage:** distance between the closest pair of points across the two clusters.
- **Complete linkage:** distance between the farthest pair of points across the two clusters.
- **Average linkage:** average pairwise distance across the two clusters.
- **Ward linkage:** merge the pair that increases within-cluster squared error the least.

After building the dendrogram, we choose clusters by cutting the tree at a selected height. This is useful in exploratory work: we can inspect broad role families first, then cut lower for finer subtypes.

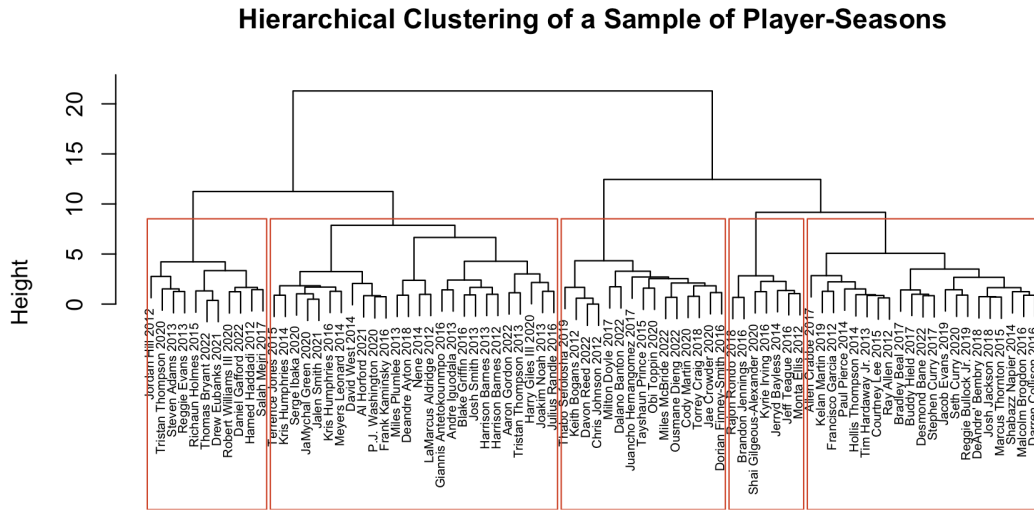


Figure 18.8: Hierarchical clustering view of a sample of NBA player-seasons.

## B Gaussian Mixture Models

K-means gives hard assignments and compact centroid-based clusters. A Gaussian mixture model gives a probabilistic version of clustering. It models the data density as a mixture of  $K$  Gaussian components:

$$f(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k),$$

where  $\pi_k \geq 0$  and  $\sum_{k=1}^K \pi_k = 1$ .

The model estimates a mean vector, covariance matrix, and mixing proportion for each component. Instead of assigning each observation to one cluster, it computes posterior membership probabilities:

$$z_{ik} = \mathbb{P}(\text{component} = k \mid \mathbf{x}_i).$$

These soft assignments are useful when a player sits between archetypes, such as a guard-wing who mixes pick-and-roll creation with movement shooting.

Gaussian mixture models are usually fit by the EM algorithm:

1. E-step: compute the current membership probabilities  $z_{ik}$ .
2. M-step: update component means, covariances, and mixing proportions using those probabilities.

The number of components can be compared with information criteria such as BIC:

$$\text{BIC}(K) = -2 \log L_K + p_K \log n,$$

where  $L_K$  is the maximized likelihood and  $p_K$  is the number of parameters.

## C Common Cluster Scores

Internal validation scores summarize cluster geometry. They are useful diagnostics, but they do not replace the original basketball question.

The silhouette coefficient for observation  $i$  is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

where  $a(i)$  is the average distance from  $i$  to observations in its own cluster and  $b(i)$  is the smallest average distance from  $i$  to observations in another cluster. Values near 1 indicate a clear assignment, values near 0 indicate a boundary case, and negative values suggest possible misassignment.

When external labels exist, we can compare a clustering to those labels with scores such as the adjusted Rand index. In a true unsupervised problem, however, the lack of labels is the point. We usually rely on geometry, stability, interpretation, and usefulness for the downstream decision.